Report on Retrieval Augmented Generation

Prajwal Kadri - pkadri

5 November 2026

1 Design and abstractions

1.1 Part 1 - General Component Description

1. I have built a RAG system to retrieve the data from the wiki corpus and answer the questions using the Gemini model. In my construction of RAG, I have created the system as follows. I have created chunks of the text similar to that in Homework 2, but built the BM25 sparse and dense retriever. Once successfully chunked, the chunks were provided to the fused retriever, which uses the Reciprocal Ranked Retriever fusion method to use both dense and sparse embedding retrievers which both retrieve 100 chunks. These RRF fused retrievers provided me with the 20 top candidates. The sparse retriever used in this project is the BM25 sparse retriever, and the dense embedding model used in this project is the all-MiniLM-L6-v2 model. Once the top 20 candidates are fetched, I re ranked the candidates using the biencoders/cross encoders and fed the top 10 into the LLM (I used Gemini here). The prompt that I created asked the LLM's to produce 5 answers at once, and I did so in order to reduce the API cost, because the cost would be increased due to the additional input token for each candidate. Once that was done, I reranked the answers using their mean token log-probability under Qwen2.5-3B-Instruct, conditioned on the prompt and excluding prompt-token scores. The Qwen-instruct model provided the raw logarithmic probability values for the prompt+candidate answer pair provided. The *score _sequence _likelihood* function calculated the scores using the Qwen model. Length normalization reduces bias toward shorter answers. Both answer rerankers preserve the original generation order when scores tie.

1.2 Part 2 - Prompts

My three prompts were designed in a way to test three different types of behavious. My third (initially first, later renamed to third) prompt was a vanilla prompt, where I provided basic instructions as required by the assignment. I asked the LLM to return the answer in a structured format and to provide a short and precise answer. In my second prompt, I added the format of the answer it is expected to provide such the format of the date and the format of the factoid question. In the first prompt(initially third, later renamed to first)

, I provided examples to the LLM, inspired by instruction tuning mechanisms. My prompts are as below:

P3 = "You are a helpful assistant, provide the answer to the question below question using the contexts provided contexts. If you are not able to find the answers in the given context, For unsupported candidates, set "answer_candidate" to "unanswerable" and "sources" to []. Return 5 valid JSON objects with these keys: - "answer_candidate": a short answer, without citations in the answer text. Match it with the answer in context. - "sources": a list of exact article titles supporting this answer. Return a JSON array containing exactly 5 objects. Always return a JSON array that contains exactly five objects. For each unsupported candidate, use: "answer_candidate": "unanswerable", "sources": [] Return raw JSON without Markdown code fences. Include only titles from the supplied contexts that you used. List each title at most once. Do not include every context title automatically."

P2 = "Provide the answer to the question below question using the contexts provided contexts. If you are unable to find the answers in the given context, For unsupported candidates, set "answer_candidate" to "unanswerable" and "sources" to []. Provide the date of the if the questions requests in the following format: MM/DD/YYYY. Short answers are returned with just one word, Long answers are returned with one sentence or two. Return 5 valid JSON object with these keys: - "answer_candidate": a short answer, without citations in the answer text. - "sources": a list of exact article titles supporting this answer. Return a JSON array containing exactly 5 objects. For each unsupported candidate, use: "answer_candidate": "unanswerable", "sources": [] Return raw JSON without Markdown code fences. Include only titles from the supplied contexts that you used. List each title at most once. Do not include every context title automatically."

P1 = "You are a assistant who provides the !!exact answer!! to what the user using the contexts provided !!contexts!!, provide the answer to the question below question. If you are not able to find the answers in the given context, For unsupported candidates, set "answer_candidate" to "unanswerable" and "sources" to []. Return 5 valid JSON object with these keys: Strictly follow the structure below: - !!"answer_candidate": It needs to be a single answer or very very small answer.!! Example: If the context says T̈he capital of France is Paris̈, and the question is Ẅhat is the capital of France?̈, the answer_candidate should be P̈aris̈. - "sources": a list of exact article titles from the supplied contexts. Example: If a context has title "France" and text "The capital of France is Paris", the answer_candidate should be "Paris" and the sources should be ["France"]. List each cited title only once. Return a JSON array containing exactly 5 objects. For each unsupported candidate, use:"answer_candidate": "unanswerable", "sources": [] Return raw JSON without Markdown code fences. Please follow the structure to the bone. Do not change the JSON structure."

Prompt 1 performed the best because there were examples from which the model could learn and it behaved somewhat like an instruction tuning of the model.

2 Main Results

2.1 Results

Context reranker	Prompt	Answer reranker	EM	Mean precision	Mean recall
Bi-encoder	p1	Likelihood	0.4349	0.7796	0.6958
Bi-encoder	p2	Likelihood	0.4236	0.7905	0.7026
Bi-encoder	p3	Likelihood	0.3892	0.8605	0.7648
Bi-encoder	p1	NumSources	0.4875	0.8563	0.7627
Cross-encoder	p1	Likelihood	0.4388	0.8054	0.7140
Cross-encoder	p1	NumSources	**0.5037**	**0.8943**	**0.7901**

Table 1: Results on all 4,681 questions using the final prompt labels. NumSources = number of supporting contexts. Source precision and recall are averaged over answerable questions only.

2.2 Per type breakdown

Question type	Questions	EM	Mean precision	Mean recall
Factoid	1147	0.7018	0.9238	0.9355
Complex	1185	0.4203	0.9505	0.9595
Multi-hop	1165	0.3554	0.8081	0.4745
Unanswerable	1184	0.5414	—	—

Table 2: Per-type results for the best run: cross-encoder with prompt p1 and Number of supporting contexts answer reranking. Source precision and recall are not applicable to unanswerable questions.

3 Comparison

The difference in metrics for each of the comparison is given below:

Comparison	ΔEM	ΔPrecision	ΔRecall
p1 − p2 (bi, likelihood)	+1.13	−1.09	−0.68
p1 − p3 (bi, likelihood)	+4.57	−8.09	−6.89
Cross − bi (likelihood)	+0.38	+2.58	+1.82
Cross − bi (NumSources)	+1.62	+3.79	+2.73
NumSources − likelihood (bi)	+5.26	+7.68	+6.69
NumSources − likelihood (cross)	+6.49	+8.89	+7.60

Table 3: Metric differences in percentage points, computed from unrounded official scores. NumSources denotes reranking by the number of supporting contexts. Context comparisons and answer-reranker comparisons use p1.

3.1 P1 vs P2 vs P3

Keep the bi-encoder fixed to perform the calculations, the P1 achieved the highest Exact matches. However, P3 achieved the highest source precision and recall among the three prompts, showing that higher answer accuracy did not go hand in hand with better source metrics. I believe P1 achieved the highest EM because its explicit short-answer instructions and worked example encouraged extracting the requested information rather than providing a broader description. **For example**, for question `qa_f26_004770`, "When was Barry Bonds born?", P1 returned "July 24, 1964," whereas P3 described his playing career instead. This is consistent with P1 following the question and answer-format instructions more closely. However, P2 returned the equivalent date "07/24/1964," which the scorer rejected.

3.2 Bi-encoder vs Cross Encoder

The Bi-encoder along with prompt 1 performed poorly as compared to the cross-encoder along with prompt 1. The Cross encoder has a better exact match, precision and mean recall compared to the bi-encoder. The answer reranking after retrieving the answers with the cross-encoder gave the best results compared to that of the Bi-encoder. I believe that both results are not surprising, as the cross encoder usually performs better, as it jointly scores both the question and the answer together. This makes the relation between the question and the chunks more prominent, and the context will be more precise, whereas in bi-encoders, we are creating the embeddings of the questions and the chunks separately and then using cosine similarity to compare the embeddings. **For example**, question `qa_f26_002566` asks who composed the choral symphony written more than eighty years after Vivaldi died in 1741. The bi-encoder selected contexts only from "Antonio Vivaldi," and both answer rerankers returned "unanswerable." The cross-encoder also selected contexts from "Symphony No. 9 (Beethoven)," and both answer rerankers returned "Ludwig van Beethoven." This example supports the explanation that the cross-encoder promoted useful evidence into the ten chunks supplied to the generator.

3.3 Liklihood vs Number of supporting contexts

Using the number of supporting contexts as an answer ranking improved the EM, precision, and recall across both retrievers. I believe this is because literal support across multiple chunks can identify the correct answer even when Qwen assigns a competing candidate higher likelihood. **For example**, "Who developed DenyHosts?" (qa_f26_003803), the likelihood selected "Jan-Pascal van Best" under both context rerankers, while using the number of supporting contexts to rerank selected the correct answer, "Phil Schwartz." In the bi-encoder contexts, "Phil Schwartz" appeared in two sections and "Jan-Pascal van Best" in one. The passage associates Jan-Pascal with independent synchronization-server software, suggesting confusion between related components.

4 Error Analysis

I analyzed three errors from our highest-EM run: cross-encoder with P1 and supporting-context-count answer reranking (EM = 0.5037).

4.1 Missing second-hop evidence: multi-hop question.

Question `qa_f26_000652` asks, "Who is identified as chair of KODE-TV's owning company?" The gold answer is "Nancie Smith," but the system returned "unanswerable." The selected context titles included "KODE-TV" but not "Mission Broadcasting," whose article identifies the chair. Importantly, "Mission Broadcasting" was also absent from the initial 20-chunk RRF pool. Therefore, the failure occurred upstream of context reranking: the cross-encoder could not select evidence that was absent from its input pool. A likely explanation is that the initial query retrieved station-related material without following the connection from KODE-TV to its owning company.

4.2 Chunk boundaries and literal matching: factoid question.

Question `qa_f26_000078` asks, "When was NY 28A built?" The gold answer is "the early 20th century," but the selected answer was "1930." The context titles show that the correct article, "New York State Route 28A," was supplied to the generator. However, the construction description was split across two 100-word chunks: one ended with "early 20th" and the next began with "century." The generated candidate "early 20th century" received a supporting-context count of zero, while "1930" appeared in two selected chunks. This illustrates how chunk boundaries and literal substring counting can favor a repeated but inappropriate date over the relevant answer.

4.3 Incomplete citations: multi-hop question.

Question `qa_f26_002566` asks who composed the choral symphony written more than eighty years after Vivaldi died in 1741. The system correctly answered "Ludwig van Beethoven," but cited only "Symphony No. 9 (Beethoven)." The gold source annotations also require "Antonio Vivaldi." Both article titles appeared in `context_titles`, so this was a citation omission rather than an article-level retrieval failure. The answer obtained EM=1 and source precision=1, but source recall was only 0.5. A plausible explanation is that the generator cited the article containing the final answer while omitting the article supporting the question's intermediate connection.